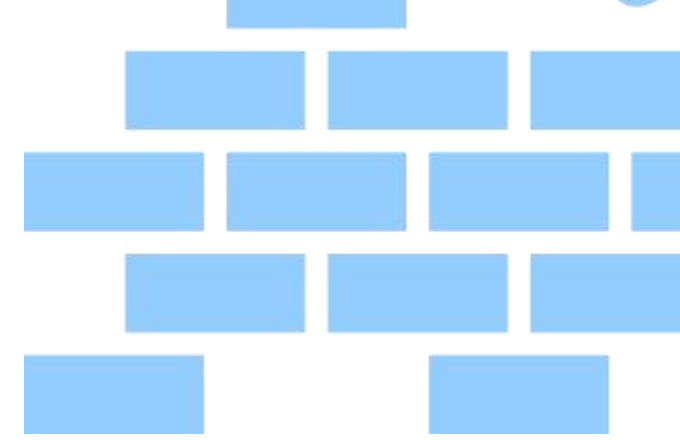
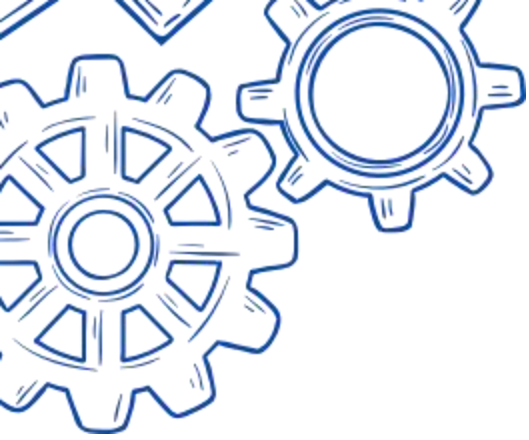




Preliminary Introduction

A Lightweight Real-Time Chat Platform

Do It Dui Team : 游翔宇、孙宇轩、邢天舒



I. Starting Point and Challenges

 Platform Solutions Resources Open Source Enterprise Pricing

socketio / chat-example Public
forked from rauchg/chat-example

<> Code Issues 2 Pull requests 1 Actions Security Insights

main 20 Branches 0 Tags

Go to file

<> Code

This branch is 13 commits ahead of, 7 commits behind rauchg/chat-example:master.

 darrachequesne fix: properly resolve relative file 07a183a · 2 years ago 13 Commits

.codesandbox	chore: add codesandbox configuration	2 years ago
.gitignore	feat: initial commit for step #7	2 years ago
LICENSE	feat: initial commit for step #1	2 years ago
README.md	docs: add README	2 years ago
index.html	feat: initial commit for step #8	2 years ago
index.js	fix: properly resolve relative file	2 years ago
package-lock.json	feat: add initial commit for step #9	2 years ago
package.json	feat: add initial commit for step #9	2 years ago

README MIT license

The MIT License (MIT)

Copyright (c) 2014-2023 Guillermo Rauch

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN

About

Basic chat example with Socket.IO

[socket.io](#)

Readme

MIT license

Activity

Custom properties

1.4k stars

42 watching

1.3k forks

Report repository

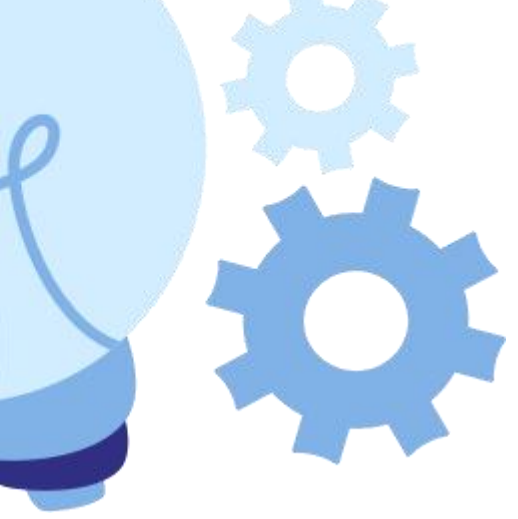
Languages

JavaScript 51.8%

HTML 47.9%

Dockerfile 0.3%

<https://github.com/socketio/chat-example/tree/main>

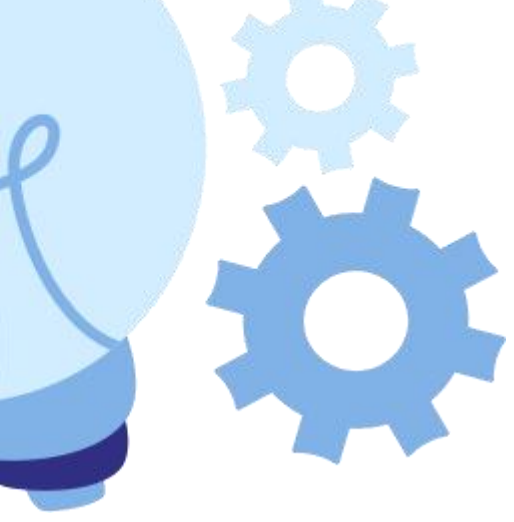


I. Starting Point and Challenges

Our backend uses **Node.js + Express + Socket.IO** for real-time communication, with messages persisted to SQLite, the frontend is vanilla **HTML/CSS/JavaScript**.

This stack lets us quickly validate the core send/receive functionality for real-time messaging.



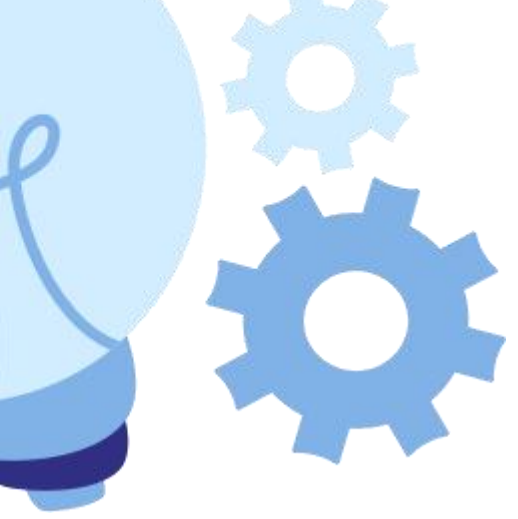


I. Starting Point and Challenges



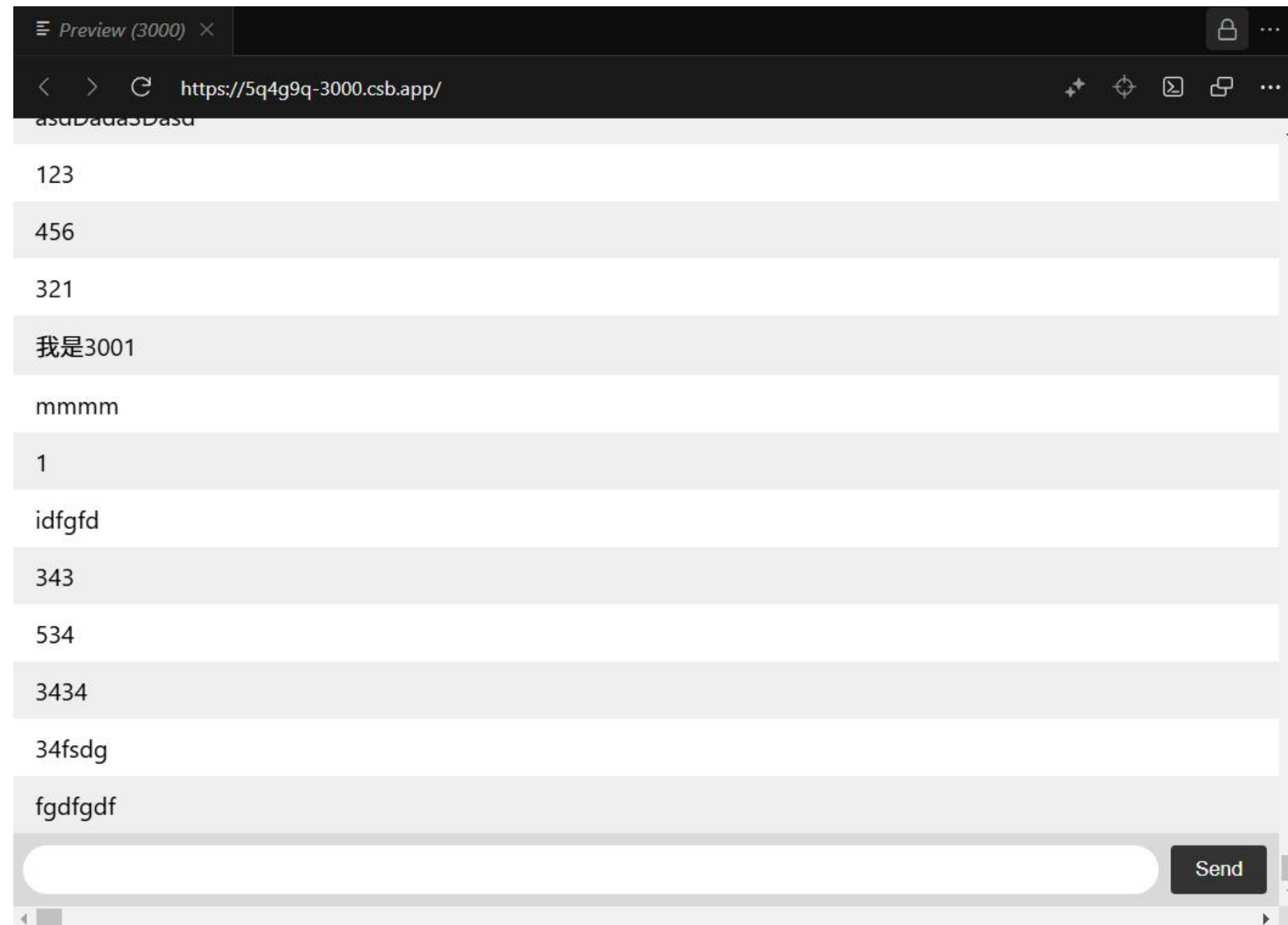
**However, as a “sample”, it is still far from
a truly “usable” product.**

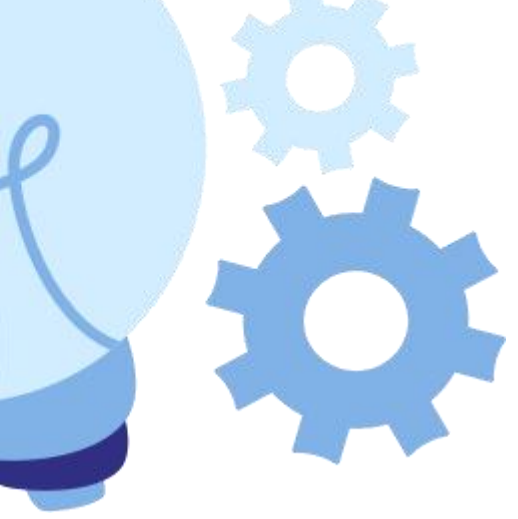




I. Starting Point and Challenges

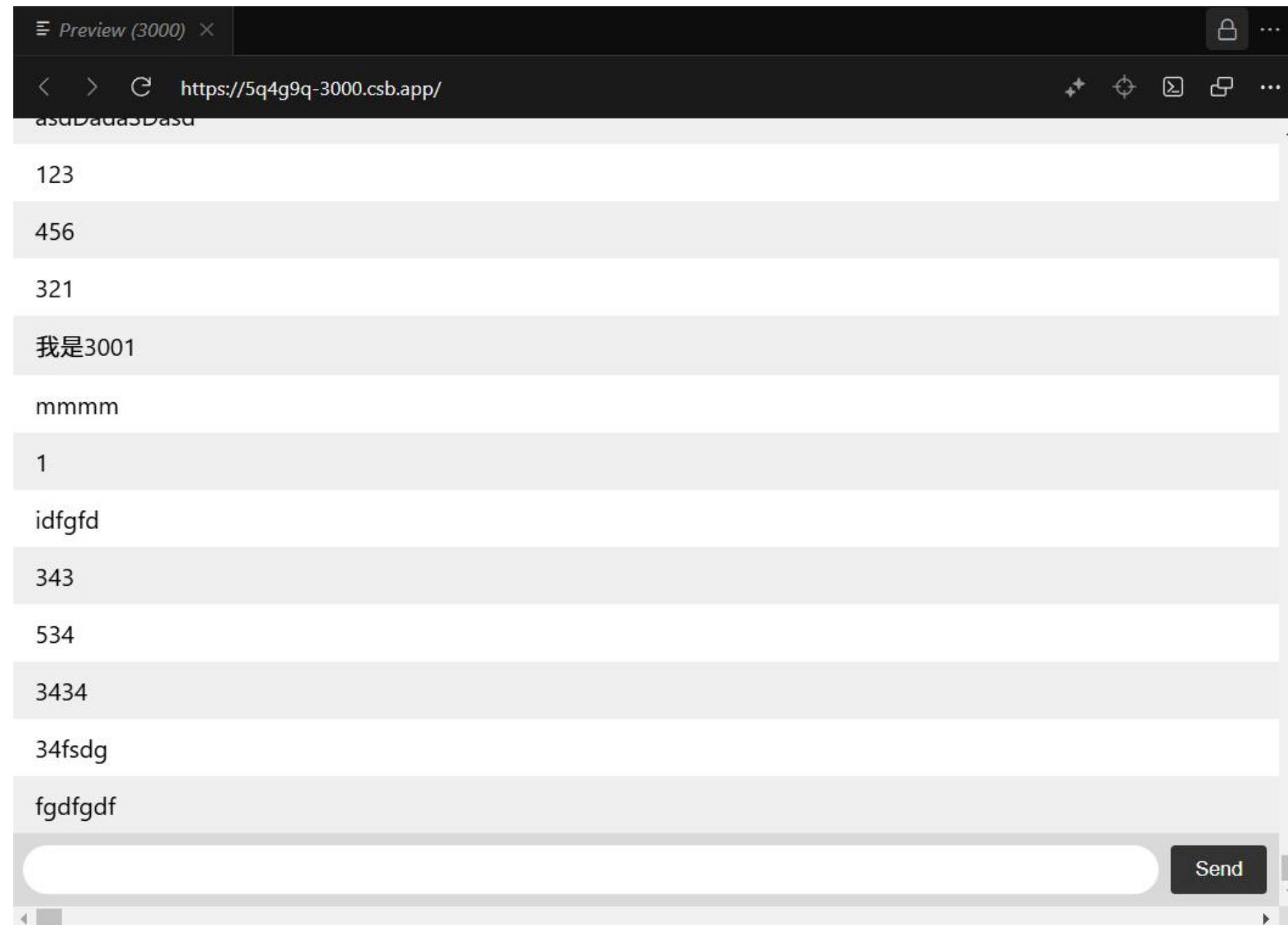
Extremely limited functionality: Only a single global public chatroom is supported. Everyone is in the same space; grouping or private spaces are not possible.





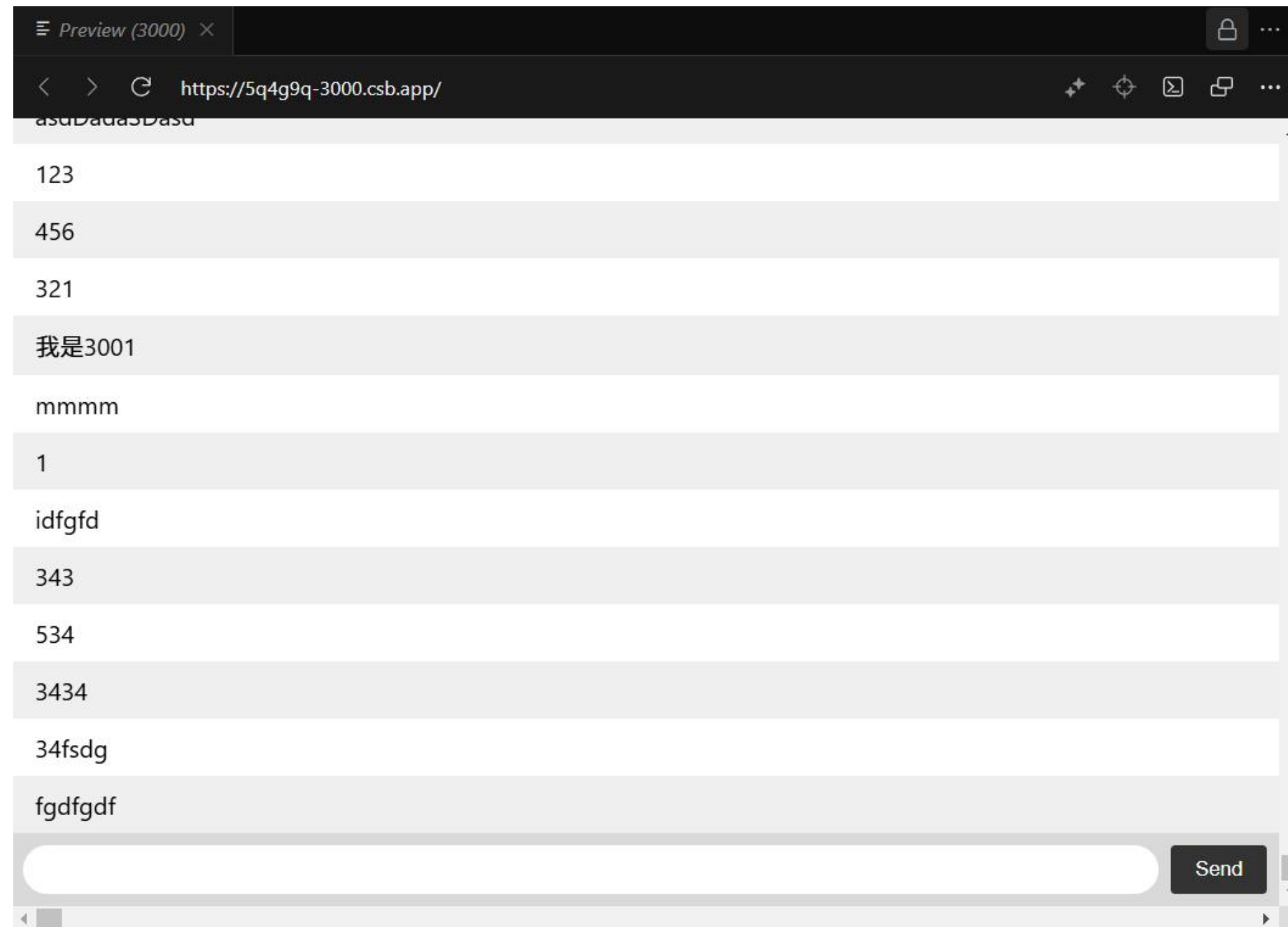
I. Starting Point and Challenges

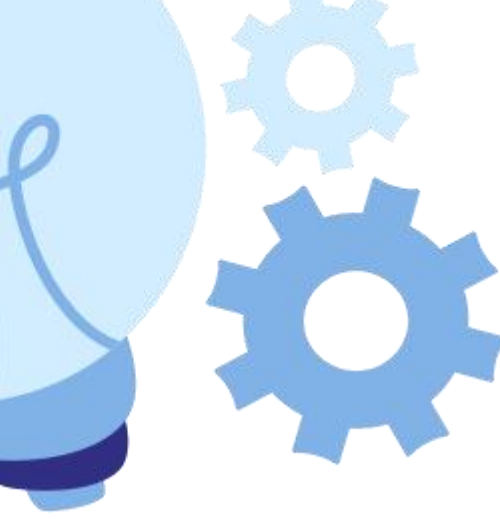
Missing user experience basics: It is fully anonymous with no nicknames or avatars, making it hard to distinguish speakers. Messages are plain text without timestamps or other basic metadata.



I. Starting Point and Challenges

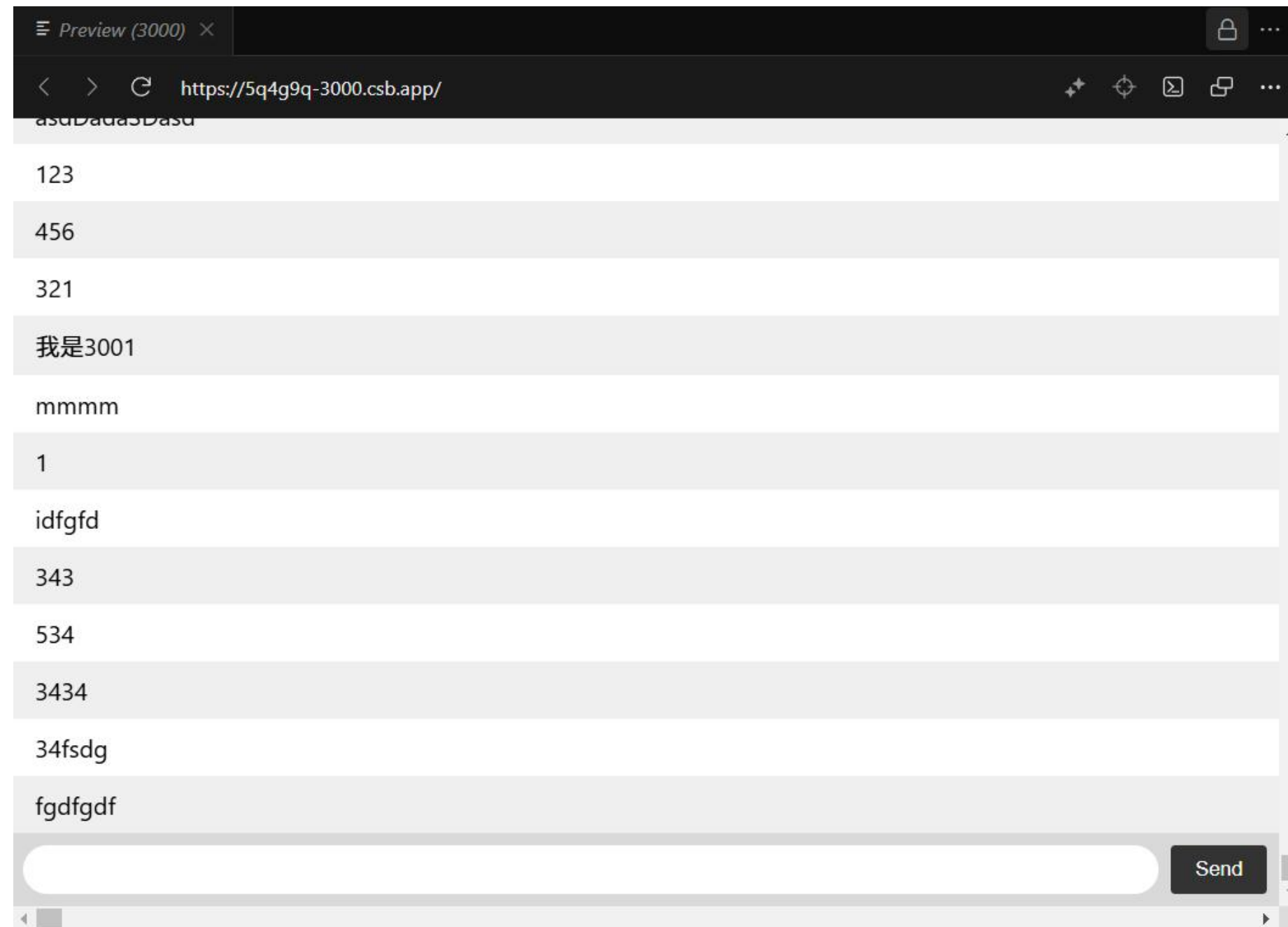
Overly bare UI: There is only a basic input box and message list with very rudimentary interactions.



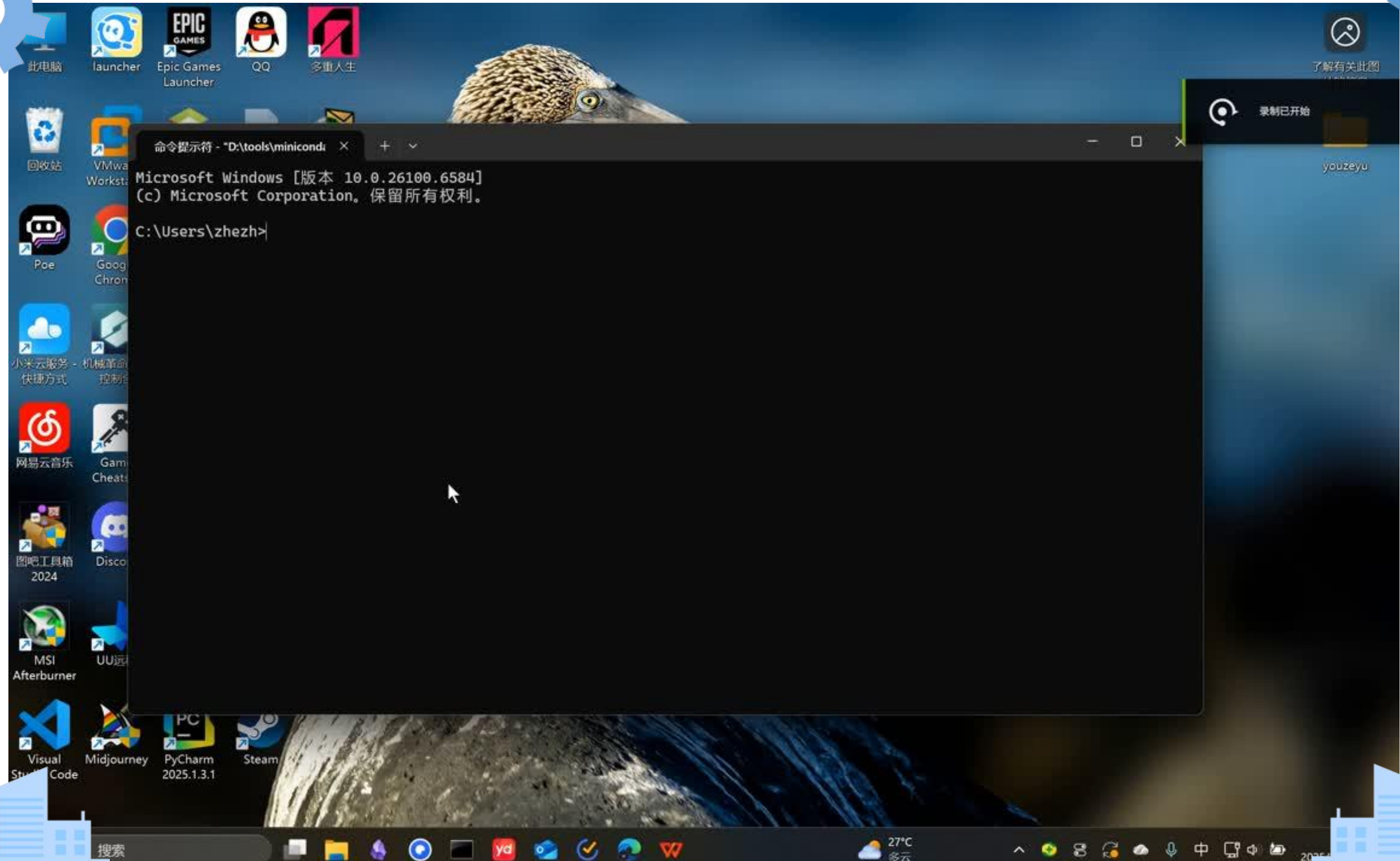


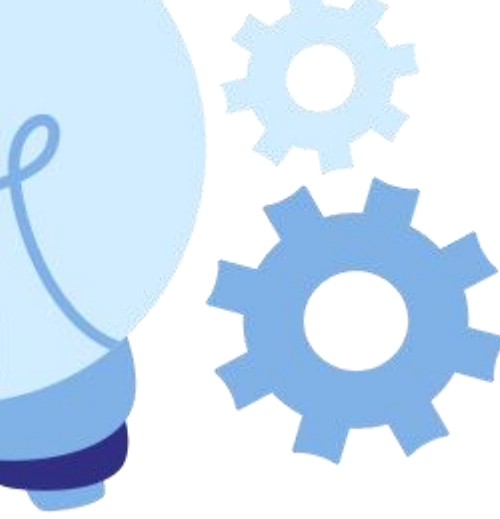
I. Starting Point and Challenges

Security and moderation gaps: There is no access control, no room passwords, and no admin roles. It is easy for malicious spamming to disrupt normal use for all users.



II. Our Solution: From “Sample” to “MVP”





II. Our Solution: From “Sample” to “MVP”

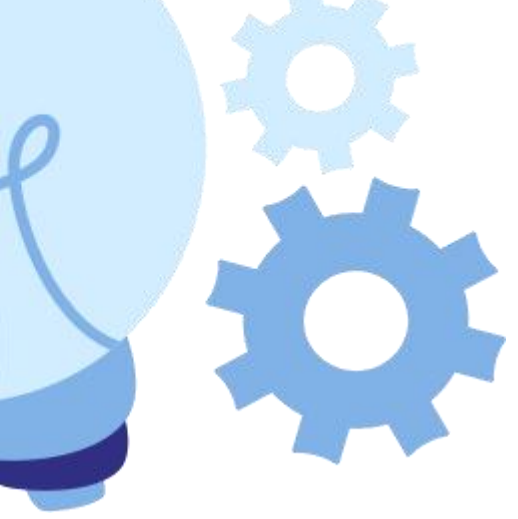
Core capability upgrades

- Multi-room support
- User identity and online user list
- Message history viewing

Deployment and usability improvements

- One-click startup and cross-network access
- Minimal learning curve

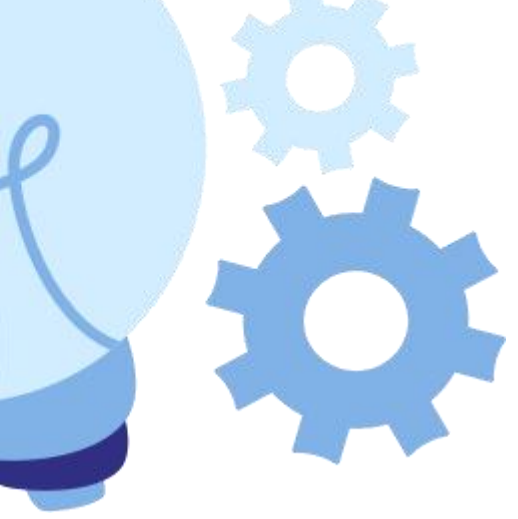




II. Our Solution: From “Sample” to “MVP”

Engineering robustness

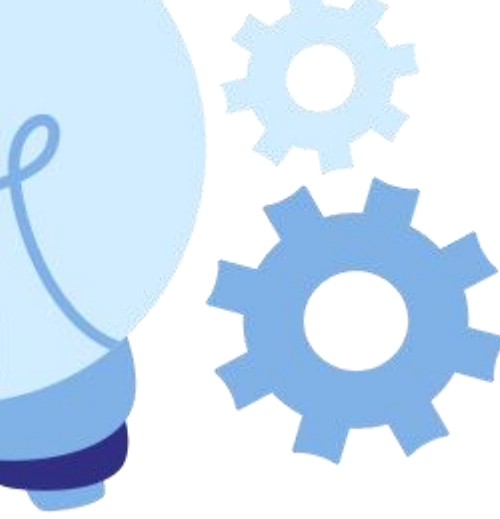
- Reliability: We added a health-check endpoint (/health) and a graceful shutdown mechanism to ensure stable, dependable service.
 - Portability: Built on Node.js, the project can be easily deployed on Windows, macOS, and Linux, or any environment that supports Node.js.
 - Documentation completeness: A detailed README provides run instructions, test cases, and an FAQ, enabling anyone to reproduce the setup quickly.
- 
- 



III. Implementation Path and Team Roles

To deliver our MVP, we focused on the following:

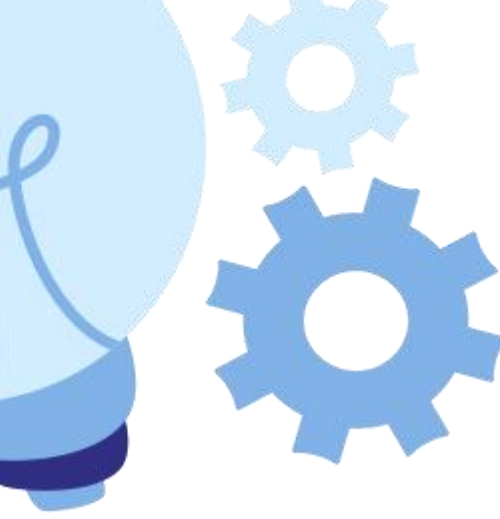
- **Backend architecture extension:** Redesigned the room and user models so the server can manage multiple independent conversation spaces concurrently.
 - **Data storage upgrade:** Refactored the SQLite schema, designing rooms and messages tables and adding indexes to optimize history queries.
 - **API and protocol alignment:** Added REST APIs such as message-history retrieval and standardized the Socket.IO interaction flow between frontend and backend.
 - **Frontend experience overhaul:** Rebuilt the UI from the ground up, adding a sidebar, navigation bar, and dark theme to significantly lower onboarding friction.
 - **Deployment and operations support:** Packaged startup scripts, integrated optional ngrok-based NAT traversal, and implemented a graceful exit mechanism.
- 
- 



IV. Outlook and Future Plans

Thanks to its out-of-the-box readiness, broad applicability, and data staying under your control, this project offers distinctive value in today's environment. Whether it's internal communication for a small shop, or scenarios with unstable public internet such as vessels and remote outposts, it provides a lightweight and reliable solution.

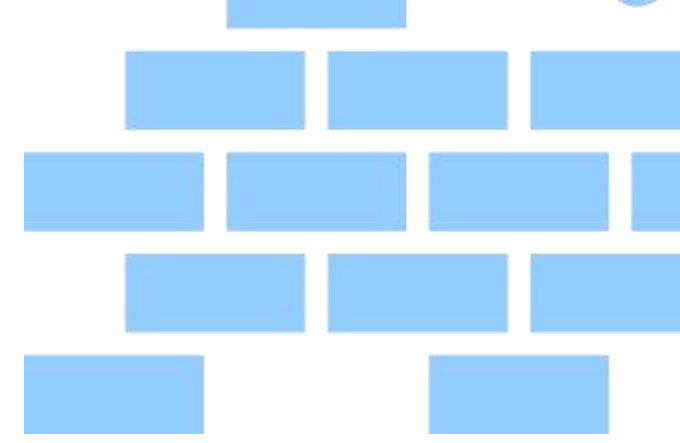
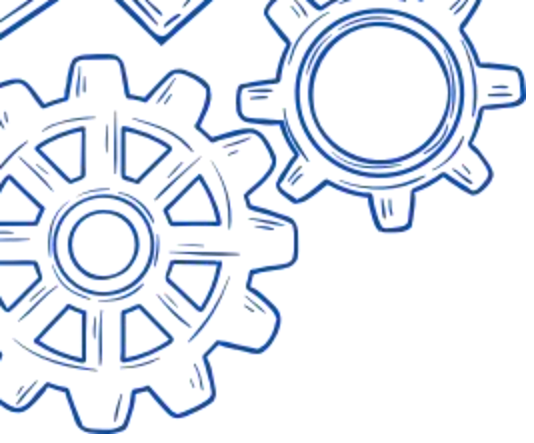




IV. Outlook and Future Plans

Future directions:

- **Enhanced security:** Add room-level password protection to meet privacy needs.
 - **Permissions model:** Introduce owner/admin roles with capabilities such as member management and muting.
 - **Richer message types:** Support additional formats such as images and files.
- 
- 



Thank You

